

The SokoLink Codebase — a guided tour

Who this is for: anyone who needs to understand how the code fits together before changing it — including us, in six weeks. `BUILD_LOG.md` says *what happened and why*, chronologically. This says *what exists now and how it works*. Current as of 2026-08-09 · **145 tests passing** · P0 complete, P1 in progress.

1. The thirty-second version

A seller connects their TikTok. We scrape their videos, an AI reads the product and price off each one, and the seller confirms. Buyers browse the result and pay by M-Pesa without leaving WhatsApp.

The hard part is the price. Kenyan sellers don't put it in the caption — measured, 0 of 10 — so we read it from the cover image, and failing that, we *listen* to the video. That cascade is the heart of the system.

2,147 lines	application code	(23 files)
1,394 lines	tests	(145 tests)
599 lines	spikes	(throwaway probes against real APIs)
283 lines	migrations	(4, each reviewed before applying)

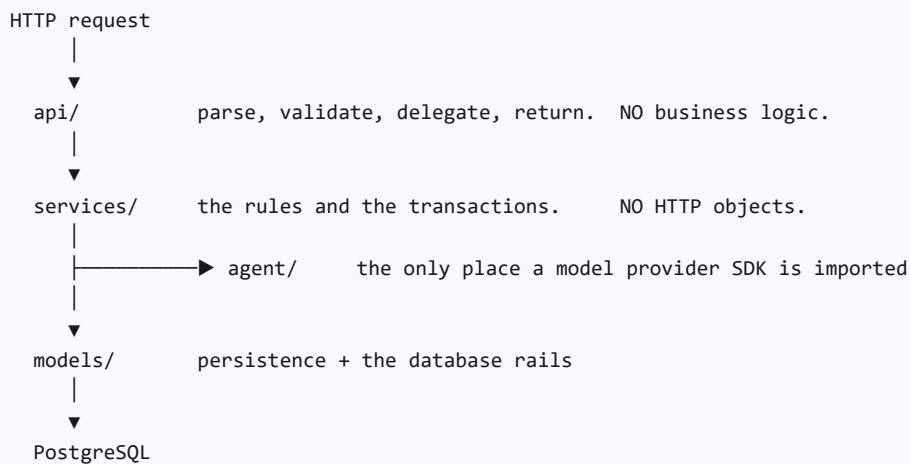
2. Running it

```
cd backend
.venv\Scripts\activate          # Windows

uvicorn app.main:app --reload    # http://localhost:8000
pytest                          # 145 tests
ruff check . && mypy app tests    # the quality gate
alembic upgrade head             # apply migrations
```

Two databases are required, and they must differ — see §7.

3. The layering rule



`schemas/` sits sideways: Pydantic types used at both the HTTP boundary *and* as the contract handed to the LLM.

A route containing business logic is in the wrong layer. A service building an HTTP response is in the wrong layer. That is the whole convention.

4. File by file

The spine

FILE	LINES	WHAT IT DOES
<code>app/main.py</code>	22	Builds the FastAPI app, registers routers. Owns nothing else — when it passes ~100 lines something is misplaced.
<code>app/config.py</code>	142	Typed settings. The only file that reads the environment.
<code>app/db.py</code>	53	Engine, session factory, <code>Base</code> . Never build an engine elsewhere.
<code>app/security.py</code>	129	Argon2id hashing and JWT sessions. Nothing else touches a password.

Three details in these that are easy to undo by accident:

- `config.py` **normalises the database driver**. Providers hand out bare `postgresql://` URLs; SQLAlchemy reads that as `psycopg2`, which we don't install. `_with_psycopg_driver()` rewrites it so `.env` holds the provider's URL verbatim on every environment.
- `config.py` **is lazy** (`@lru_cache`). An eager `settings = Settings()` would throw the moment anything imported the module, breaking tests and tooling.
- `db.py` **sets** `pool_pre_ping=True`. Cloud Postgres silently drops idle connections; without this the app hands a request a dead one and the user sees a 500 that vanishes on retry.

Models — where the rails live

FILE	LINES	WHAT IT HOLDS
models/enums.py	97	Platform , IngestMethod , ProductStatus , ScrapeStatus , PriceSource
models/account.py	63	A login. Email + Argon2id hash.
models/seller.py	108	A shop and its permanent public slug.
models/social_account.py	97	One connected platform account. A seller has many.
models/product.py	279	A catalogue item. The largest and most constrained file.
models/scrape_job.py	97	One ingestion run, and the cache that keeps Apify affordable.

Why enums are strings + CHECK constraints, not native PG enums: adding a value to a native enum needs a migration and an exclusive lock. Adding one here is a code change. Platforms will grow.

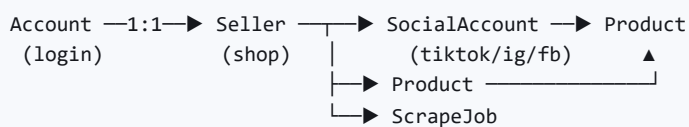
Ingestion

FILE	LINES	WHAT IT DOES
schemas/tiktok.py	121	The validation border. Apify JSON → validated objects.
services/scraper.py	208	Apify behind <i>our</i> interface. ScraperEngine is a Protocol.
schemas/draft.py	113	The contract the vision model must satisfy.
agent/draft.py	171	The provider seam. The only file that knows we use Gemini.
services/drafting.py	124	The cascade. Where the economics live.

Accounts

FILE	LINES	WHAT IT DOES
services/accounts.py	188	Signup, login, slug reservation. Anti-enumeration is the point.

5. The data model



Provenance is two dimensions, not one

This trips people up, so it is worth stating plainly:

COLUMN	ANSWERS	VALUES
platform	Where it came from	tiktok · instagram · facebook · jumia · manual
ingest_method	How it got here	profile_sync · single_link · upload

They are separate because **only the second decides re-sync ownership**. A feed sync owns what it created and may update it. It must never touch a product the seller uploaded by hand — a seller who adds stock, syncs, and watches it vanish does not come back.

`Product.is_sync_owned` is that guard, and it has its own test.

The rails — constraints in Postgres, not in service code

A model, a future agent, a migration script and a hand-written query all have to obey the database. Only application code has to *remember* to call a validator.

CONSTRAINT	PREVENTS
published_requires_price	the AI pushing an unpriced item live
stock_non_negative	overselling, at the storage layer
price_positive	KSh 0 — always a parse failure, never a giveaway
price_plausible ($\leq 10m$)	a phone number misread as a price reaching a storefront
uq_products_platform_post	a re-scrape duplicating instead of updating
upload_has_no_post_id	an uploaded product looking re-syncable
manual_iff_upload	an incoherent "tiktok upload"
unit_quantity_and_label_together	"KES 3,000 for 30" — of what?
published_needs_whatsapp	a live shop nobody can contact
slug_format	a URL a buyer cannot type
failed_needs_error	a scrape failure nobody can debug

Most of the model tests assert Postgres *refuses* something. A constraint nobody has watched fail is only a claim.

Money

`price_kes` is an **integer**, and it is the price of **one unit as sold** — which is not always one item.

```
price_kes      = 3000
unit_quantity  = 30
unit_label     = "pairs"
price_display  → "KES 3,000 for 30 pairs"
```

Found by spiking a real seller, not by design: @zumamitumbabales sells mitumba **bales**. Rendering a bare "KES 3,000" would make a buyer expect one pair. Use `price_display`, never `price_kes` directly, in anything a buyer sees.

6. Two flows, traced

A. A TikTok handle becomes draft products

1. `ApifyEngine.fetch_profile("@zumamitumbabales")`
services/scraper.py – runs the actor, caps at 30 recent posts
2. `TikTokVideo.from_apify(item)` × N
schemas/tiktok.py – THE BORDER. Flattens nested videoMeta, normalises hashtags, validates. A changed payload fails HERE, loudly, not three layers down as a None.
3. `DraftingService.draft_for_video(video)`
services/drafting.py – the cascade:
 - └ tier 2: download cover → `DraftAgent.draft_from_cover()`
 - confident price? — yes → STOP. Done.
 - └ no → escalate
 - └ tier 3: download clip → `DraftAgent.draft_from_video()`
Gemini WATCHES and LISTENS.
4. `ProductDraft (validated) → Product (status=DRAFT)`
Never published. A human confirms.

Where the money is spent: step 1 (Apify, ~\$0.30/1,000 posts) and steps 2–3 (Gemini). Tier 3 is the expensive one and, for the seller we spiked, the only one that worked — 0/10 captions, 0/4 covers, 3/3 videos.

Where the safety is: step 4. Everything the model produces is a draft.

B. A seller signs up

1. `create_account(email, password, shop_name)`
services/accounts.py
2. `validate → hash (Argon2id) → reserve_slug()`
"Nairobi Thrift" → "nairobi-thrift"
taken? → "nairobi-thrift-2" (never blocks signup)
reserved word? → refused (no seller takes /login)
3. Account + Seller created in ONE transaction
A login with no shop is a dead end.
4. `create_session_token(account.id) → signed JWT → HttpOnly cookie`

Login is deliberately paranoid. When no account matches, it *still* runs an Argon2 verification against `DUMMY_HASH`. Without that, a missing email returns almost instantly while a real one pays for hashing — a timing difference measurable over the network, and a free account-enumeration oracle. There is a test asserting the unknown-email path is not an order of magnitude faster.

Unknown email, wrong password, and deactivated account all return **one identical message**.

7. Tests

FILE	TESTS	PROTECTS
test_models.py	~45	The database rails. Mostly asserts Postgres refuses .
test_auth.py	43	Hashing, sessions, slugs, anti-enumeration
test_scraper.py	26	The validation border, against the real captured payload
test_drafting.py	~20	The cascade — which tiers ran , i.e. cost
test_config.py	14	Settings, hermetic (<code>_env_file=None</code>)
test_health.py	4	Liveness vs readiness

Three conventions worth knowing:

- **Externals are always mocked.** No test calls Apify, Gemini, Meta or Daraja. They cost money and rate-limit.
- **Scraper fixtures come from the real payload** captured by spike 01. A test passing against a payload we imagined proves nothing.
- **Cascade tests assert which tiers ran**, not only the output. A regression that quietly escalated every item would pass a naive correctness test and arrive as a bill.

Two databases, on purpose

`TEST_DATABASE_URL` must be set and must **differ** from `DATABASE_URL` .

- Unset → database-backed tests **skip** (a visible gap).
- Identical → the suite **refuses to run**.

The `_schema` fixture **drops and recreates**, because `create_all` only adds missing tables and never alters existing ones — that drift once produced 25 failures reading *"column unit_quantity does not exist"*.

8. Spikes

`spikes/` holds throwaway probes against real APIs. Nothing in `app/` imports them, and they are kept because their **findings** justify the design.

SPIKE	COST	FOUND
01 · Apify profile	paid	Payload shape. 0/10 captions have prices. Bio carries phone numbers.
02 · Offline analysis	free	No transcriptions. Cover URLs expire. Clips are 9–24s.
03 · Gemini cover	paid	0/4 — and the model correctly refused to invent a price.
04 · Gemini video	paid	3/3 . And <code>price_evidence</code> revealed bulk pricing.

Spike 01 writes its raw response to `spikes/out/` , so every later question is answered offline for free. **Spend once, analyse many times.**

9. Where to start reading

In this order — each builds on the last:

1. `app/models/product.py` — the domain, and every rail. Start here.
2. `app/schemas/draft.py` — what we demand of the AI, and what we deliberately *don't* let it decide.
3. `app/services/drafting.py` — the cascade. The clearest expression of how cost shapes this system.
4. `app/services/accounts.py` — read `authenticate()` closely; the anti-enumeration reasoning is non-obvious.
5. `tests/test_models.py` — the rails, proven. Reads as a specification.

Every file opens with a header docstring saying what it does, the pipeline it sits in, and **why it is shaped that way**. If something looks strange, that docstring almost certainly explains it — and if it doesn't, that is a documentation bug worth fixing.

10. Not built yet

	STATUS
Ingestion service — persist drafts, once-per-day guard	next
Manual upload path	P1
API routes for the three ingestion paths	P1
Seller dashboard (Jinja2 + HTMX)	P3
Buyer storefront	P3
WhatsApp channel	P2 — <i>gated on Meta verification</i>
Orders + M-Pesa	P4
Soko AI, Soko Intel	P5–P8

The scraper, the agent and the cascade all work and are tested. **Nothing yet connects them to an HTTP request or writes a Product to the database** — that is the ingestion service, and it is the next thing built.